Programmatic Modeling of High-Order Observers

Zixi Wang

Xi'an Jiaotong University



Contents

1. Problem Setting
2. Basic Knowledge
3. Order-2 Observer Pipeline
4. Theorem
5. Example
6. Complexity and Summary
7. Future Work

Problem Setting

Consider a finite-state system G observed by two agents.

- Agent A_1 estimates the current state of G from its observations.
- Agent A_2 has a different observation of the same execution.
- Agent A_2 infers the possible state estimates held by Agent A_1.

Models

$$G = (Q, E, \delta, Q_0), \qquad A_i = (E_i, \Sigma_i, \ell_i),$$

where $E_i \subseteq E$ and $\ell_i : E \to \Sigma_i \cup \{\epsilon\}$ maps events outside E_i to ϵ.

Target computation

For an observation β of Agent A_2, compute

$$M_{A_1 \leftarrow A_2}(G, \beta) = \{M_1(G, \ell_1(s)) : s \in L(G),\ \ell_2(s) = \beta\}.$$

Labeled Finite-State Automaton

An LFSA is a sextuple

$$S = (Q, E, \delta, Q_0, \Sigma, \ell),$$

where Q is the state set, E is the event alphabet, $\delta : Q \times E \to 2^Q$ is the transition function, $Q_0 \subseteq Q$ is the initial-state set, Σ is the output alphabet, and

$$\ell : E \to \Sigma \cup \{\epsilon\}$$

is the labeling function.
For an observed word α, the current-state estimate is

$$M(S, \alpha) = \{q \in Q : \exists q_0 \in Q_0, \exists s \in E^*,\ \ell(s) = \alpha,\ q \in \delta(q_0, s)\}.$$

Source: Zhang, *Annual Reviews in Control*, 56, 100902, 2023, Definition 1.

Observer

For $S = (Q, E, \delta, Q_0, \Sigma, \ell)$, its observer is

$$S_{\text{obs}} = (Q_{\text{obs}}, \Sigma_{\text{obs}}, \delta_{\text{obs}}, q_{0,\text{obs}}),$$

where

$$Q_{\text{obs}} = 2^Q, \qquad \Sigma_{\text{obs}} = \ell(E) \setminus \{\epsilon\}.$$

Define the unobservable reach

$$UR(X) = \bigcup_{q \in X} \bigcup_{s \in E_{uo}^*} \delta(q, s).$$

Then

$$q_{0,\text{obs}} = UR(Q_0),$$

$$\delta_{\text{obs}}(X, a) = UR\left(\bigcup_{q \in X} \bigcup_{\substack{e \in E_o \\ \ell(e) = a}} \delta(q, e) \right).$$

Source: Zhang, *Annual Reviews in Control*, 56, 100902, 2023, Definition 3.

Concurrent Composition

For $S_i = (Q_i, E_i, \delta_i, Q_{0i}, \Sigma, \ell_i)$, $i = 1, 2$, define

$$CC_A(S_1, S_2) = (Q', E', \delta', Q_0', \Sigma, \ell'),$$

$$Q' = Q_1 \times Q_2, \qquad Q_0' = Q_{01} \times Q_{02}.$$

Observable events synchronize when their labels agree:

$$E_o' = \{(e_1, e_2) \in E_{1o} \times E_{2o} : \ell_1(e_1) = \ell_2(e_2)\}.$$

Unobservable events interleave:

$$E_{uo}' = \{(e_1, \epsilon) : e_1 \in E_{1uo}\} \cup \{(\epsilon, e_2) : e_2 \in E_{2uo}\}, \qquad E' = E_o' \cup E_{uo}'.$$

The transition relation satisfies

$$((q_1, q_2), (e_1, e_2), (q_1', q_2')) \in \delta' \iff (q_1, e_1, q_1') \in \delta_1 \wedge (q_2, e_2, q_2') \in \delta_2,$$

with the unchanged component retained for each unobservable interleaving. Moreover,

$$\ell'((e_1, e_2)) = \ell_1(e_1) = \ell_2(e_2), \qquad \ell_i(\epsilon) = \epsilon.$$

Source: Zhang, *Annual Reviews in Control*, 56, 100902, 2023, Definition 2.

Order-2 Observer Pipeline

1. Construct Agent A_1's observer

$$G + A_1 \longrightarrow G_{A_1}, \qquad Obs_{\ell_1}(G) =: Obs_{A_1}.$$

`fa_agent_to_lfsa.py` `observer.py`

2. Compose and change the observation perspective

$$CC_{\ell_1}(G_{A_1}, Obs_{A_1}) \xrightarrow{(e_1,e_2)\mapsto e_1,\ \ell_1\mapsto\ell_2} CC^{G,Obs}_{A_1\to A_2}.$$

`CC.py` `lfsa_to_fa.py` `fa_agent_to_lfsa.py`

3. Construct the order-2 observer

$$Obs_{\ell_2}\left(CC^{G,Obs}_{A_1\to A_2}\right) =: Obs_{A_1\leftarrow A_2}(G).$$

`observer.py`

Source: Zhang et al., *High-Order Observers and High-Order State-Estimation-Based Properties*, Slide 35.

Theorem

If reading β leads the order-2 observer to the state

$$\{(q_1, X_1), \ldots, (q_n, X_n)\},$$

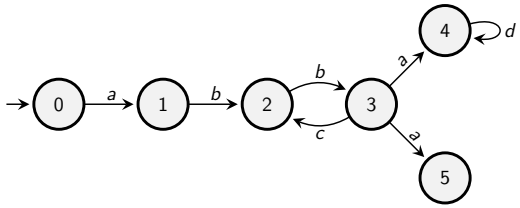
then

$$M_{A_1 \leftarrow A_2}(G, \beta) = \{X_1, \ldots, X_n\}.$$

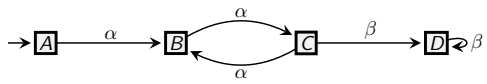
Each pair (q_i, X_i) records a possible actual state q_i and a corresponding estimate X_i held by Agent A_1. Therefore, projecting the second components gives Agent A_2's inference about Agent A_1.

Source: Zhang et al., *High-Order Observers and High-Order State-Estimation-Based Properties*, Theorem 25.

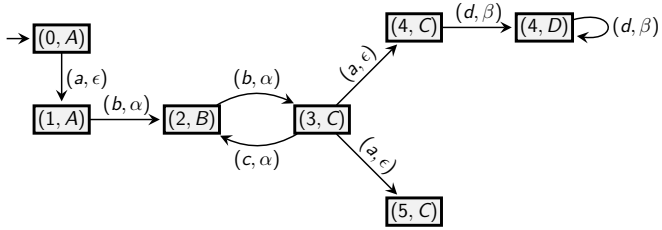
Example



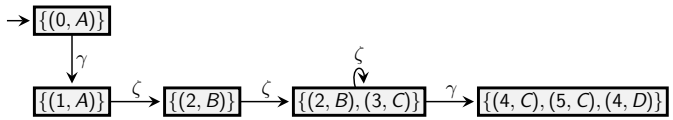
(a) Automaton G, where $E_1 = \{b, c, d\}$, $\ell_1(b) = \ell_1(c) = \alpha$, $\ell_1(d) = \beta$, $E_2 = \{a, b\}$, $\ell_2(a) = \gamma$, $\ell_2(b) = \zeta$.



(b) The observer Obs_{A_1} of automaton G_{A_1}, where $A = \{0, 1\}$, $B = \{2\}$, $C = \{3, 4, 5\}$, $D = \{4\}$.



(c) The concurrent composition $\text{CC}_{\ell_1}(G_{A_1}, \text{Obs}_{A_1})$.



(d) The order-2 observer $\text{Obs}_{A_1 \leftarrow A_2}(G)$.

The implementation reproduces the published five-state, five-transition order-2 observer.

Reference figure: Zhang et al., *High-Order Observers and High-Order State-Estimation-Based Properties*, Example 27.

Complexity and Summary

Let $n = |Q|$. In the worst case,

$$|Obs_{A_1}| \leq 2^n, \qquad |CC| \leq n2^n, \qquad |Obs_{A_1 \leftarrow A_2}| \leq 2^{n2^n}.$$

Therefore, explicit construction has a double-exponential upper bound in both time and storage, although only reachable states are generated.

Summary

- The programs implement LFSA conversion, observers, and concurrent composition.
- The resulting order-2 observer computes Agent A_2's possible inferences about Agent A_1.
- The published example provides an end-to-end validation case.

Future Work

- Verify properties of Agent A_2's inference about Agent A_1's state estimates.
- Extend the pipeline to construct order-n observers.
- Develop visualization tools for generated observers and intermediate models.